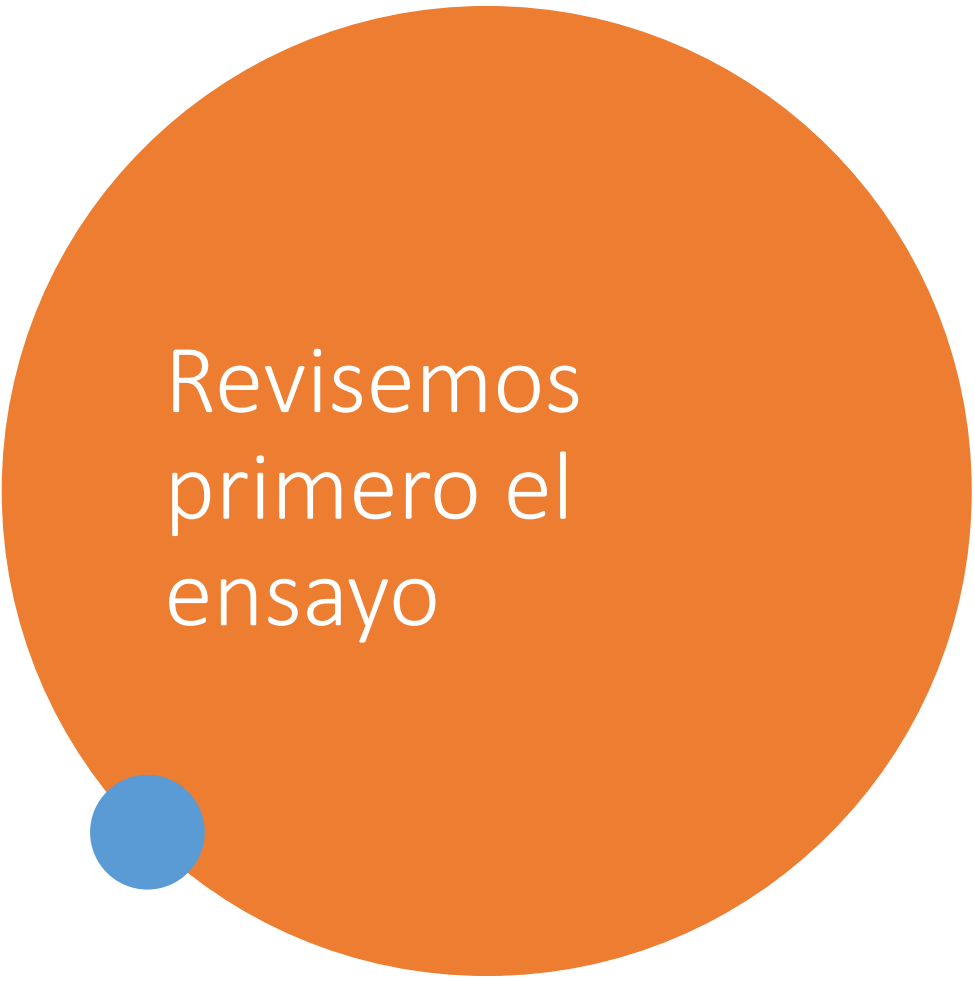


Arquitecturas Avanzadas de Software

Universidad EAFIT
Semana 2
2025



Revisemos
primero el
ensayo

- 
- Conversación activa,
 - Aprender a escuchar,
 - Todos queremos que nos escuchen y no sabemos escuchar a los demás.

Kafka vs. RabbitMQ: Duelo de Mensajería

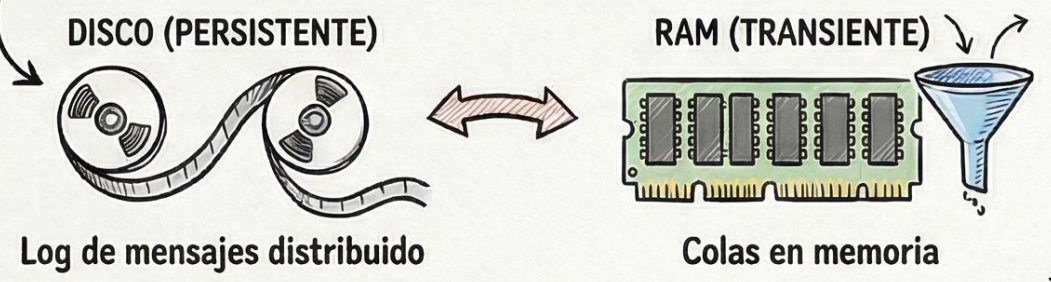
Comparando arquitecturas, rendimiento y casos de uso ideales. Elección: ¿Volumen masivo o flexibilidad?



ARQUITECTURA Y RENDIMIENTO



Log vs. Colas de Mensajes



Retención y Replay



Rendimiento vs. Latencia



COMPARATIVA OPERATIVA RÁPIDA

	Kafka	RabbitMQ
Escalabilidad	✓ Excelente (vía particiones)	⚠ Moderada (requiere lógica extra)
Enrutamiento	✓ Básico (por tópicos)	⚠ Muy sofisticado (exchanges AMQP)
Protocolos	✓ Propietario	✓ Estándares (AMQP, MQTT, STOMP)

Casos ideales para Kafka



¿CUÁL ELEGIR PARA TU PROYECTO?

Uso Combinado



RabbitMQ para procesamiento inmediato, luego Kafka para almacenamiento a largo plazo. ¡El mejor de ambos mundos!

Casos ideales para RabbitMQ



PATRONES DE ARQUITECTURA



Soluciones comprobadas para construir sistemas **robustos, escalables y mantenibles**



CRECIMIENTO EVOLUTIVO COMPLEJO (COMO LA SAGRADA FAMILIA)

Evoluciona por etapas y se adapta a nuevas necesidades sin perder estabilidad.



API GATEWAY

Punto único de entrada que gestiona, enruta y protege las solicitudes a los servicios.



BASE ESTABLE Y ROBUSTA (COMO LA PARROQUIA ITALIANA)

Cimientos sólidos que garantizan confiabilidad, seguridad y rendimiento a largo plazo.



KAFKA /
RABBITMQ

KAFKA /
RABBITMQ



EVENT BROKER

KAFKA / RABBITMQ

Gestiona eventos en tiempo real para comunicación desacoplada entre servicios.



PATRÓN CQRS LECTURA (READ)

Optimiza consultas para obtener información rápida, consistente y escalable.



ESCRITURA (WRITE)

Gestiona cambios de estado de forma segura y eficiente.

MICROSERVICIOS Y BASES DE DATOS POLÍGLotas



INFRAESTRUCTURA HÍBRIDA



Los patrones de arquitectura son **soluciones reutilizables** basadas en experiencia que guían el diseño de sistemas flexibles, resilientes y alineados con los **objetivos del negocio**.





Patrones/Estilos de arquitectura



Los patrones y estilos guían **decisiones arquitectónicas** para construir soluciones escalables, mantenibles y alineadas al negocio.



Soluciones comprobadas



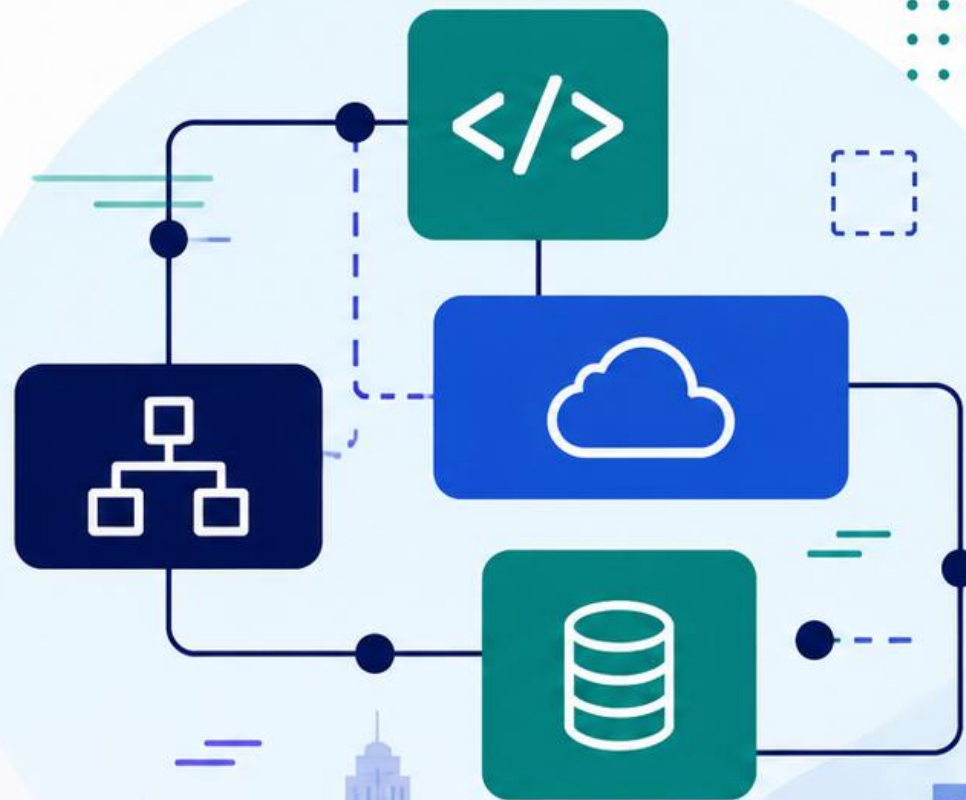
Eficiencia y escala



Calidad y consistencia



Alineación con el negocio



Relación Estilo – Patrón (Style – Pattern)



Estilo Arquitectónico (Architectural Style)



Estilo Arquitectónico (Architectural Style)

- Un **estilo arquitectónico** define la **estructura organizativa global** de un sistema de software. Es una **familia de sistemas** que comparten ciertas **características estructurales y principios de diseño**. Un estilo arquitectónico es mucho más que un patrón, ya que dicta la **organización fundamental** de todo el sistema. Es el "**tipo**" de **arquitectura** que estás utilizando.



Patrón Arquitectónico (Architectural Pattern)



Patrón Arquitectónico (Architectural Pattern)

- Un **patrón arquitectónico** es una **solución reusable** a un **problema recurrente** en un **contexto específico** dentro del diseño de software. Piensa en él como una **plantilla probada** para resolver un desafío de diseño particular. Los patrones arquitectónicos no son arquitecturas completas por sí mismos, sino más bien **bloques de construcción o mecanismos** que puedes incorporar en tu diseño general.



Estilo Arquitectónico

(Architectural Style)



Estilo Arquitectónico

(Architectural Style)

Un estilo arquitectónico define la **estructura organizativa global** de un sistema de software.

Es una **familia de sistemas** que comparten **características estructurales y principios de diseño**.

Es mucho más que un patrón, ya que dicta la **organización fundamental** de todo el sistema.

Es el "**tipo**" de **arquitectura** que estás utilizando.



Un **estilo arquitectónico** define la **estructura organizativa global** de un sistema de software. Es una **familia de sistemas** que comparten ciertas **características estructurales y principios de diseño**. Un estilo arquitectónico es mucho más que un patrón, ya que dicta la **organización fundamental** de todo el sistema. Es el "**tipo**" de **arquitectura** que estás utilizando.



Beneficios



Proporciona una visión de alto nivel del sistema



Asegura consistencia en la estructura y componentes



Facilita decisiones de diseño estratégicas



Mejora la mantenibilidad y escalabilidad



Tip: Elegir un estilo arquitectónico adecuado al contexto del negocio y a los requisitos técnicos es clave para el éxito del sistema en el largo plazo.



Un buen estilo guía el diseño, reduce la complejidad y alinea tecnología con estrategia.

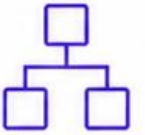


Relación Estilo – Patrón (Style – Pattern)

- **Jerarquía:** Los estilos operan a un nivel de abstracción superior que los patrones. Un estilo proporciona el contexto general dentro del cual se pueden aplicar múltiples patrones.
- **Composición:** Un sistema software puede implementar un estilo arquitectónico principal que incorpora múltiples patrones de arquitectura para resolver problemas específicos.
- **Especialización:** Los patrones a menudo representan especializaciones o refinamientos de los principios generales establecidos por un estilo arquitectónico.
- **Implementación:** Los patrones pueden ser utilizados como mecanismos concretos para implementar aspectos de un estilo arquitectónico más abstracto.
- **Los estilos arquitectónicos se construyen utilizando patrones arquitectónicos.** Un estilo arquitectónico define la organización general, pero la **implementación concreta** de ese estilo a menudo implica el uso de **múltiples patrones arquitectónicos** para resolver problemas específicos dentro de esa arquitectura.
- **Los patrones arquitectónicos pueden ser utilizados dentro de diferentes estilos arquitectónicos.** Un patrón como el "Proxy" o el "Pub-Sub" no está limitado a un único estilo arquitectónico. Puede ser útil en arquitecturas monolíticas, de microservicios, o basadas en eventos, dependiendo de las necesidades.
- **Los estilos arquitectónicos proporcionan el contexto para aplicar patrones.** El estilo arquitectónico elegido define el "marco general" dentro del cual los patrones arquitectónicos se utilizan para refinar el diseño y resolver desafíos específicos.

Diferencias Estilo – Patrón (Style – Pattern)

Aspecto	Estilo de Arquitectura	Patrón de Arquitectura
Alcance	Global, define la estructura general	Local, aborda problemas específicos
Nivel de abstracción	Mayor (macro-arquitectura)	Menor (meso-arquitectura)
Especificidad	Define principios generales	Proporciona soluciones concretas
Contexto de aplicación	Define el contexto arquitectónico	Opera dentro de un contexto dado
Adaptabilidad	Define restricciones amplias	Ofrece soluciones parametrizables
Propósito	Establece el paradigma estructural	Resuelve problemas recurrentes



Los estilos arquitectónicos se pueden clasificar según diferentes criterios. A continuación, **las clasificaciones más comunes:**



Según la Estructura del Sistema

Describe cómo se organiza el sistema en componentes y cómo interactúan entre sí.

- Capas (Layers)
- Cliente-Servidor
- Tuberías y Filtros
- Repositorio
- Microservicios



Según el Alcance de Aplicación

Indica el nivel o dominio en el que se aplica el estilo.

- Arquitectura Empresarial
- Arquitectura de Soluciones
- Arquitectura de Software



Según la Composición

Define cómo se combinan los componentes o módulos del sistema.

- **Arq. Monolíticas**
- Basada en Componentes
- Orientada a Servicios (SOA)
- Microkernel (Plug-in)



Según las Cualidades de Calidad

Se enfoca en atributos de calidad específicos que el sistema debe cumplir.

- Rendimiento
- Disponibilidad
- Seguridad
- Escalabilidad
- Modificabilidad



Según la Tecnología o Paradigma

Basado en el paradigma de programación o tecnología utilizada.

- Orientada a Objetos
- Orientada a Eventos
- Reactiva
- Serverless



Monolitos

Monolithic architecture styles are generally much simpler than distributed ones, and as such are easier to design and implement. These single deployment unit applications are fairly inexpensive from an overall cost standpoint. Furthermore, most applications architected using a monolithic architecture style can be developed and deployed much more quickly than distributed ones.

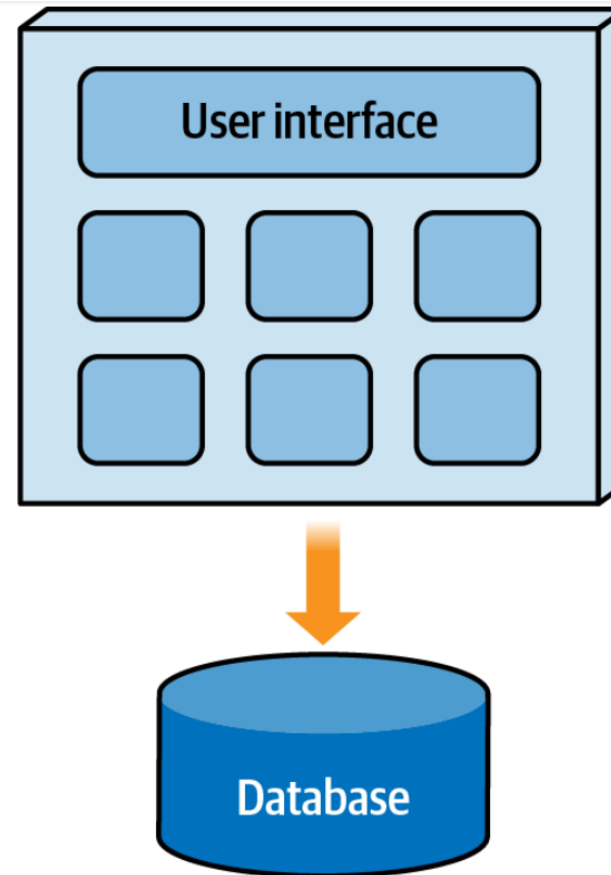


Figure 2-1. Monolithic architectures are single deployment units

Distribuidas

As the name suggests, distributed architectures consist of multiple deployment units that work together to perform some sort of cohesive business function. In today's world, most distributed architectures consist of services, although each distributed architecture style has its own unique formal name for a service.

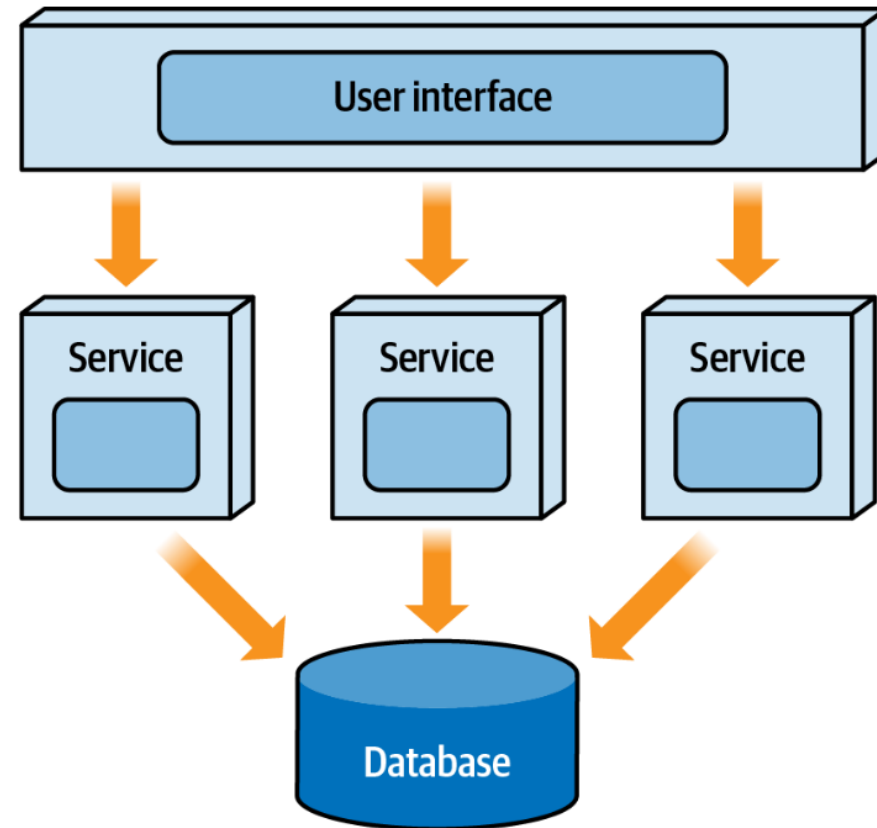


Figure 2-2. Distributed architectures consist of multiple deployment units

Particionamiento técnico

In this architecture style, components are organized by technical layers; for example, presentation components that have to do with the user interface, business layer components that have to do with business rules and core processing, persistence layer components that interact with the database, and the database layer containing the data for the system.

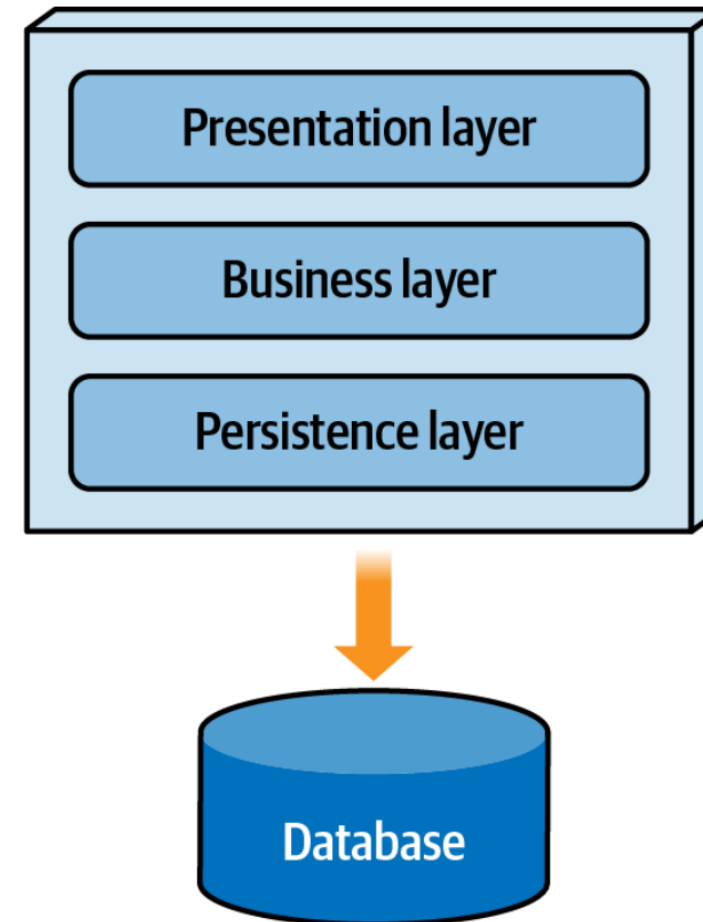


Figure 2-3. In a technically partitioned architecture, components are grouped by their technical usage

Particionamiento por dominios

Unlike technically partitioned architectures, components in domain partitioned architectures are organized by domain areas, not technical usage. This means that all of the functionality (presentation, business logic, and persistence logic) is grouped together for each domain and subdomain area in separate areas of the application.

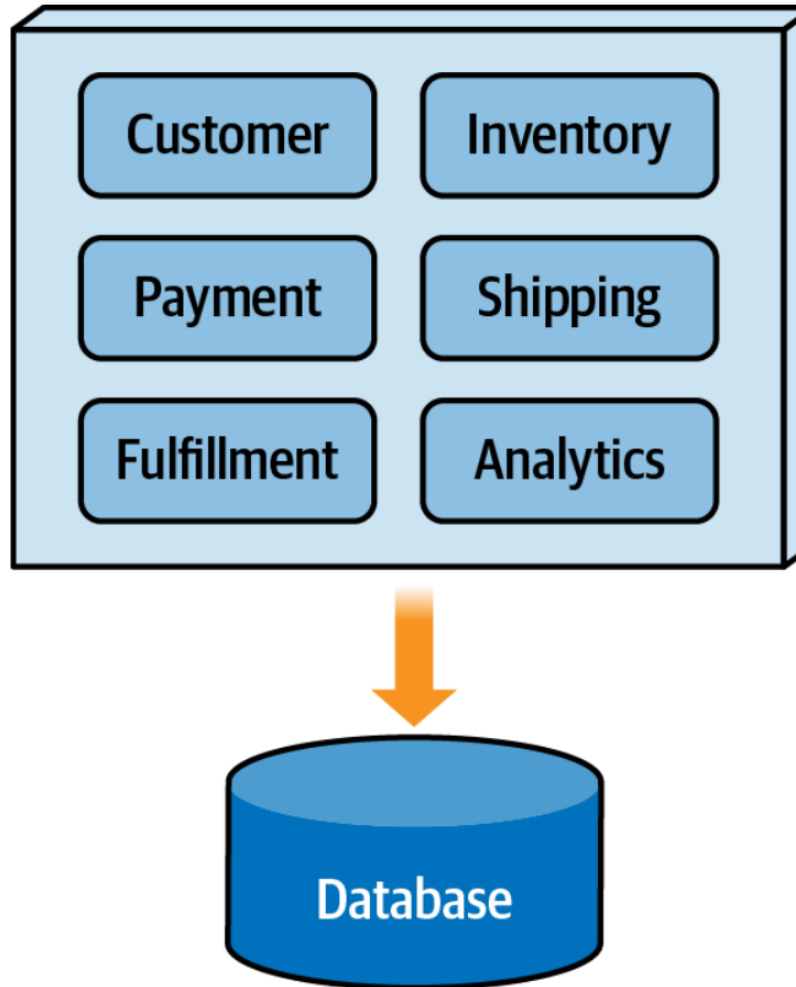


Figure 2-4. In a domain partitioned architecture, components are grouped by domain area

Particionamiento técnico – Arq x Capas

- Components within the layered architecture style are organized into horizontal layers, each performing a specific role within the application (such as presentation logic, business logic, persistence logic, and so on). Although the number of layers may vary, most layered architectures consist of four standard layers: presentation, business, persistence, and database

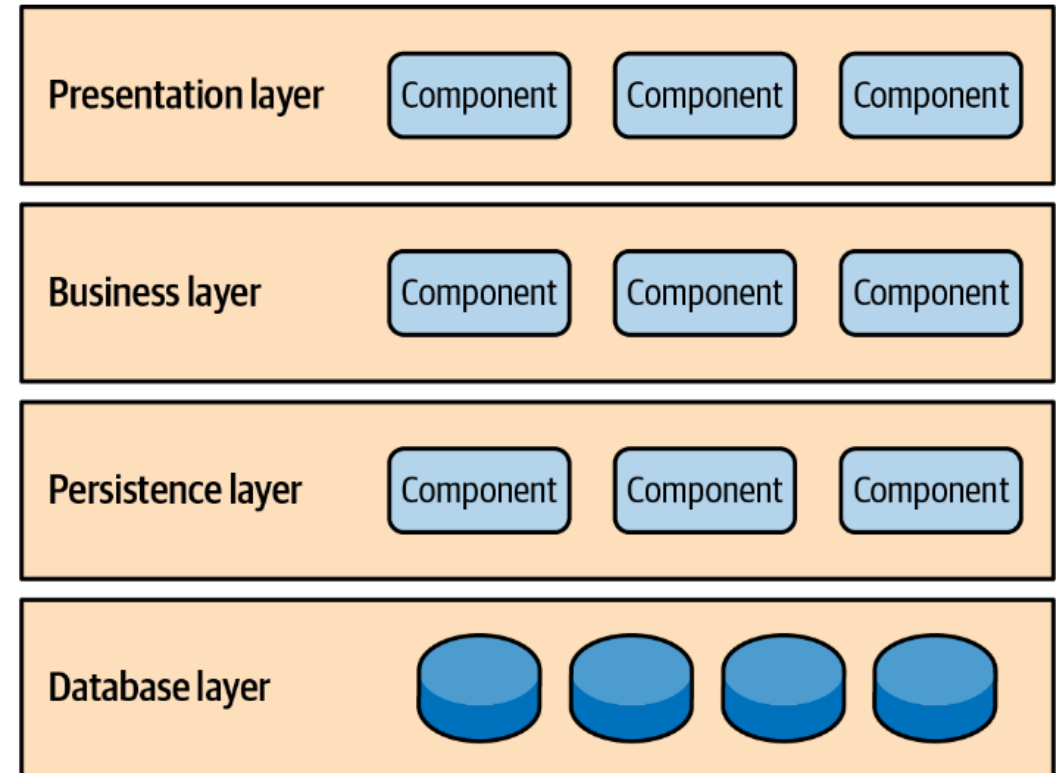


Figure 3-1. The layered architecture style is a technically partitioned architecture

Characteristic	Star rating
Partitioning type	Technical
Overall cost	\$
Agility	★
Simplicity	★★★★★
Scalability	★
Fault tolerance	★
Performance	★★★
Extensibility	★

Figure 3-5. Architecture characteristics star ratings for the layered architecture

Particionamiento técnico - Microkernel

- The microkernel architecture style is a flexible and extensible architecture that allows a developer or end user to easily add additional functionality and features to an existing application in the form of extensions, or “plug-ins,” without impacting the core functionality of the system.

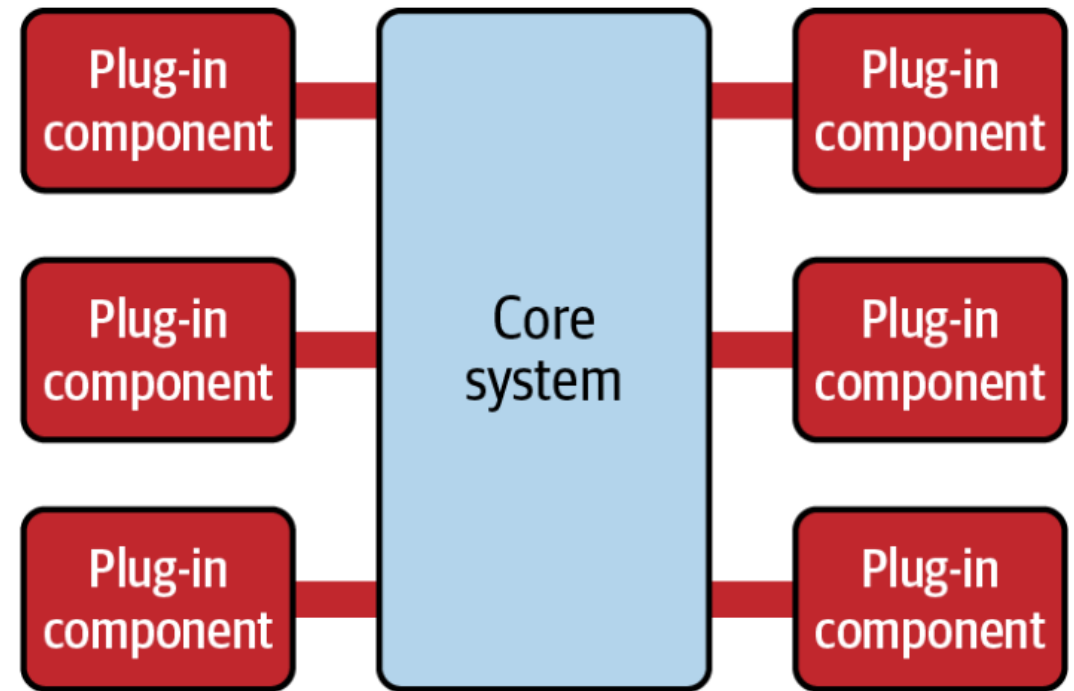


Figure 4-1. Microkernel architecture style

Characteristic	Star rating
Partitioning type	Technical or domain
Overall cost	\$
Agility	★ ★ ★
Simplicity	★ ★ ★ ★
Scalability	★
Fault tolerance	★
Performance	★ ★ ★
Extensibility	★ ★ ★

Figure 4-3. Architecture characteristics star ratings for the microkernel architecture

Particionamiento técnico – Event driven

- The event-driven architecture style has significantly gained in popularity and use over recent years, so much so that even the way we think about it has changed. This high adoption rate isn't overly surprising given some of the hard problems event-driven architecture solves, such as complex nondeterministic workflows and highly reactive and responsive systems

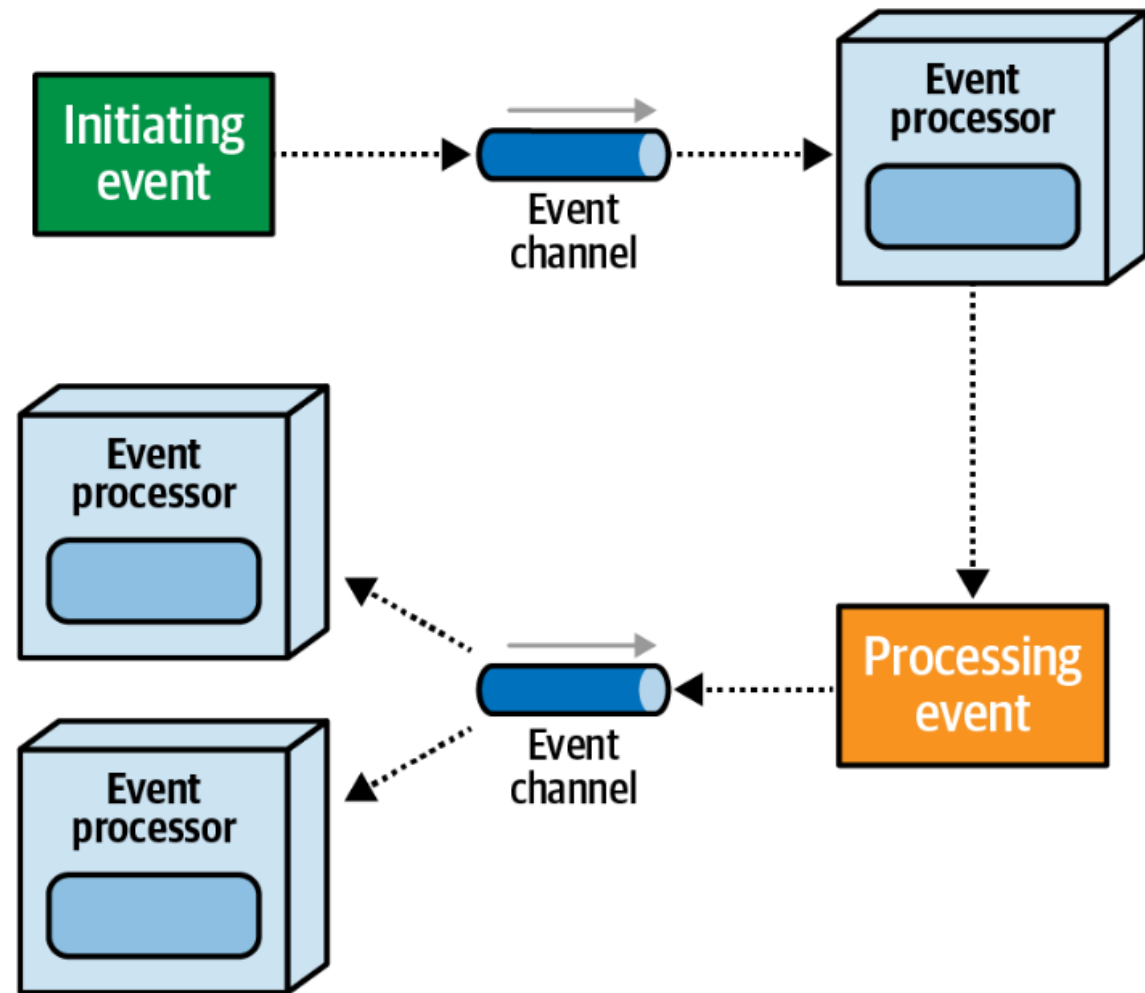


Figure 5-1. The main components of event-driven architecture

Characteristic	Star rating
Partitioning type	Technical
Overall cost	\$ \$ \$
Agility	★ ★ ★
Simplicity	★
Scalability	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Performance	★ ★ ★ ★ ★
Extensibility	★ ★ ★ ★ ★

Figure 5-5. Architecture characteristics star ratings for event-driven architecture

Particionamiento técnico – microservicios

- The microservices architecture style is an ecosystem made up of single-purpose, separately deployed services that are accessed typically through an API gateway. Client requests originating from either a user interface (usually a microfrontend) or an external request invoke well-defined endpoints in an API gateway, which then forwards the user request to separately deployed services. Each service in turn accesses its own data, or makes requests to other services to access data the service doesn't own.

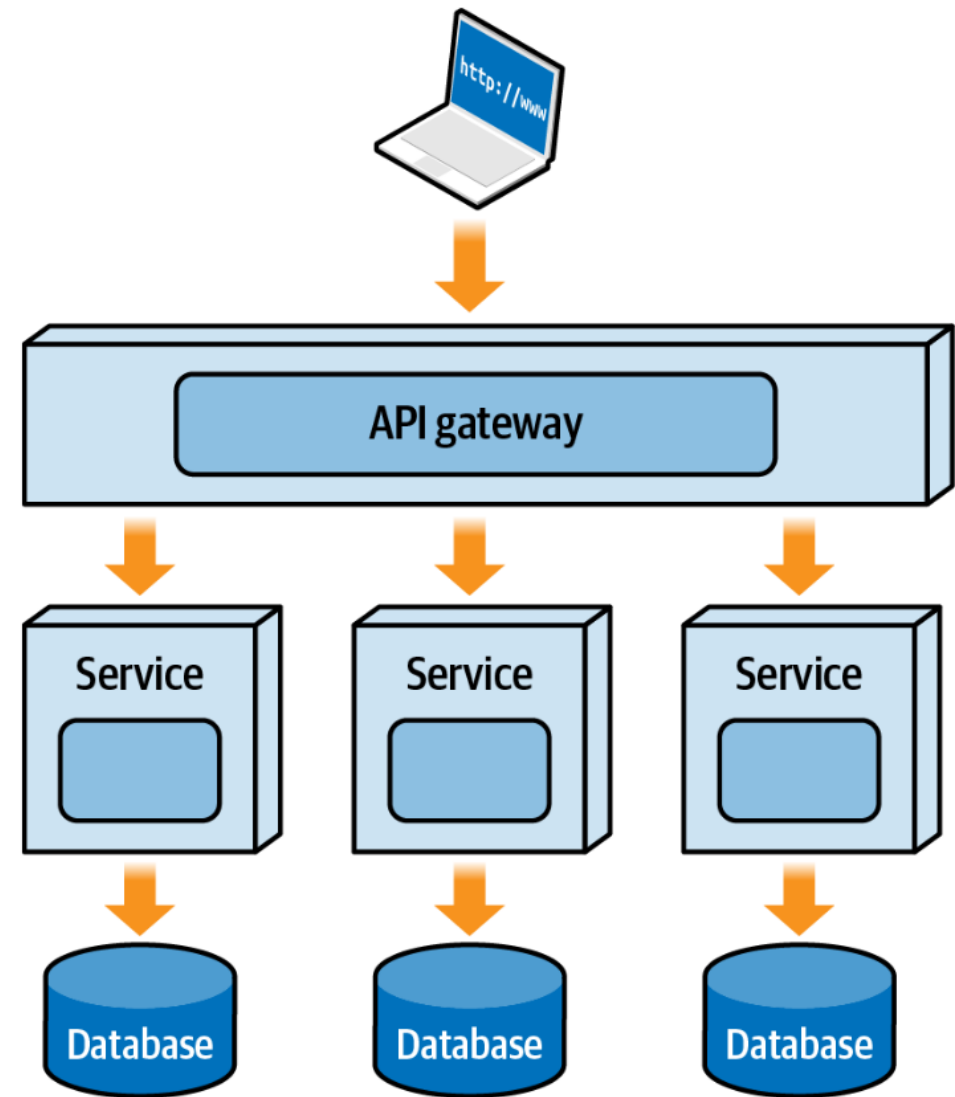


Figure 6-1. The basic topology of the microservices architecture style

Architecture Characteristics

The chart illustrated in [Figure 5-5](#) summarizes the overall capabilities (architecture characteristics) of event-driven architecture in terms of star ratings. One star means the architecture characteristic is not well supported, whereas five stars means it's well suited for that particular architecture characteristic.

Characteristic	Star rating
Partitioning type	Technical
Overall cost	\$ \$ \$
Agility	★ ★ ★
Simplicity	★
Scalability	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Performance	★ ★ ★ ★ ★
Extensibility	★ ★ ★ ★ ★

Figure 5-5: Overall capabilities of event-driven architecture

Particionamiento técnico – Space-Based Architecture

- The space-based architecture style is specifically designed to address and solve these sorts of high scalability and concurrency issues. It is also a useful architecture style for applications that have variable and unpredictable concurrent user volumes (known as elastic systems). Solving extreme and variable scalability needs is exactly what space-based architecture is all about.

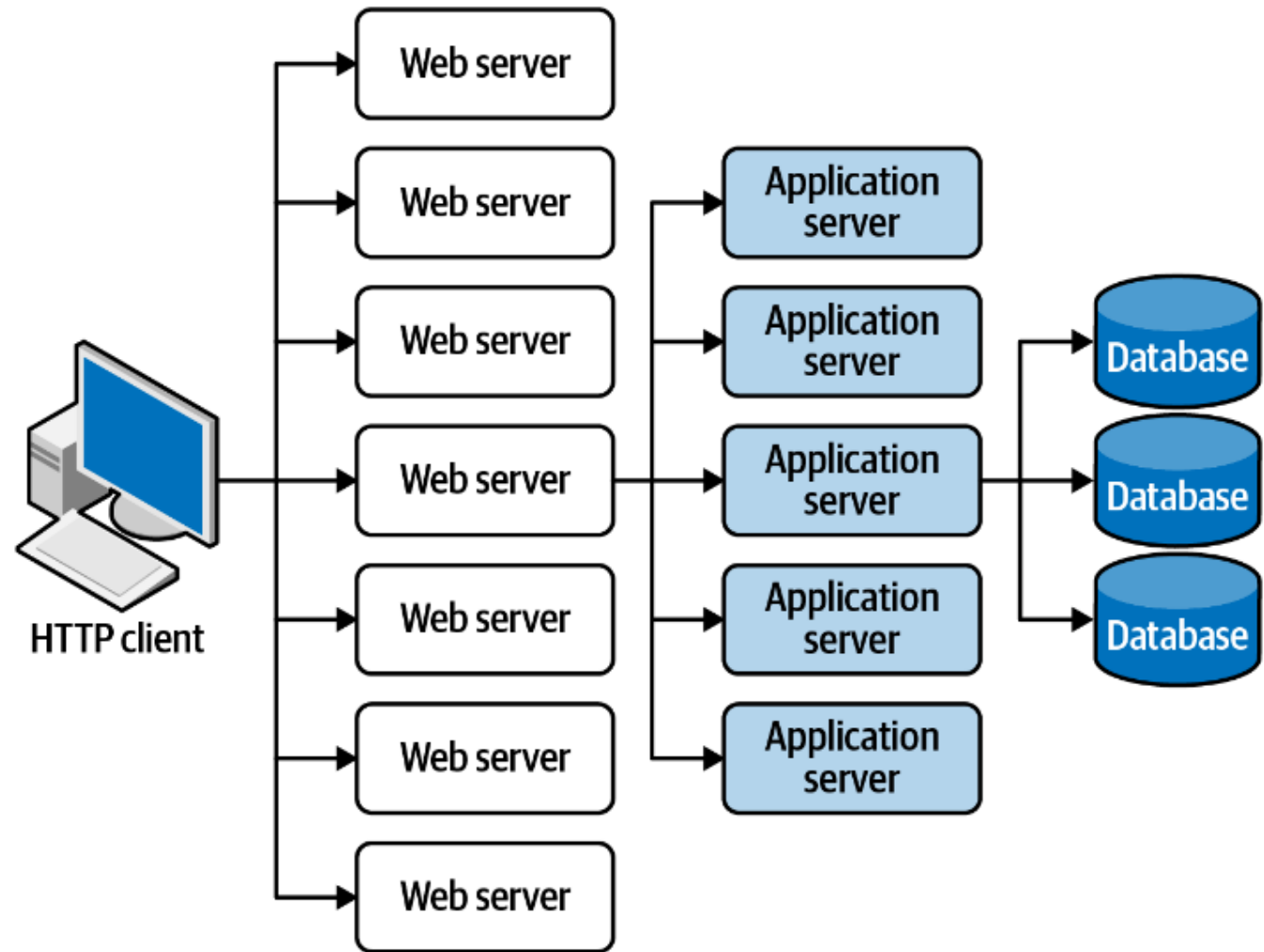


Figure 7-1. The database is usually the ultimate bottleneck for highly scalable systems

Characteristic	Star rating
Partitioning type	Technical
Overall cost	\$ \$ \$ \$ \$
Agility	★ ★
Simplicity	★
Scalability	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★
Performance	★ ★ ★ ★ ★
Extensibility	★ ★ ★

Figure 7-5. Architecture characteristics star ratings for space-based architecture

Summary

This summary will help you determine which style might be best for your situation. For example, if your primary architectural concern is scalability, you can look across this chart and see that the event-driven style, microservices style, and space-based style are probably good architecture style choices. Similarly, if you choose the layered architecture style for your application, you can refer to the chart to see that deployment, performance, and scalability might be risk areas in your architecture.

	Layered	Microkernel	Event-driven	Microservices	Space-based
Partitioning	T	D/T	T	D	T
Overall cost	\$	\$	\$\$\$	\$\$\$\$\$	\$\$\$\$\$
Agility	●	●●●	●●●	●●●●●	●●
Simplicity	●●●●●	●●●●●	●	●	●
Scalability	●	●	●●●●●	●●●●●	●●●●●
Fault tolerance	●	●	●●●●●	●●●●●	●●●
Performance	●●●	●●●	●●●●●	●●	●●●●●
Extensibility	●	●●●	●●●●●	●●●●●	●●●

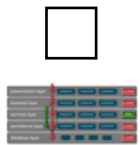
Figure A-1. Architecture styles rating summary

Architecture Styles Worksheet

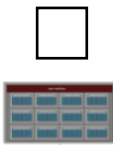
System/Project: _____

Architect/Team: _____ Date: _____

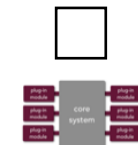
Selected Architecture(s):



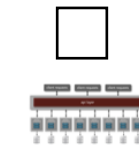
layered



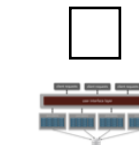
modular monolith



microkernel



microservices



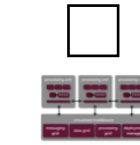
service-based



service-oriented



event-driven



space-based

partitioning	technical	domain	domain	domain	domain	technical	technical	technical
cost	\$	\$	\$	\$\$\$\$\$	\$\$	\$\$\$\$	\$\$\$	\$\$\$\$
maintainability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
testability	★★	★★	★★★	★★★★★	★★★★★	★	★★	★
deployability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
simplicity	★★★★★	★★★★★	★★★★★	★	★★★	★	★★	★
scalability	★	★	★	★★★★★	★★★★	★★★★	★★★★★	★★★★★
elasticity	★	★	★	★★★★	★★★	★★★★	★★★★	★★★★★
responsiveness	★★★★	★★★★	★★★★	★★	★★★★	★★	★★★★★	★★★★★
fault-tolerance	★	★	★	★★★★★	★★★★★	★★★★	★★★★★	★★★
evolvability	★	★	★★★★	★★★★★	★★★★★	★	★★★★★	★★★
abstraction	★	★	★★★★	★	★	★★★★★	★★★★★	★
interoperability	★	★	★★★★	★★★★	★★	★★★★★	★★★★	★★

Ejercicios

Sistema de Gestión de Inventarios

Solución Necesaria: Una aplicación web para gestionar el inventario de una tienda minorista, con módulos para entrada/salida de productos, reportes y auditorías.

Descripción del Problema: El sistema debe ser modular, fácil de mantener, y permitir la separación clara entre la lógica de negocio, la interfaz de usuario y el acceso a datos.

Por qué usar Capas: El patrón de Capas organiza el sistema en niveles (presentación, lógica de negocio, datos), promoviendo la separación de responsabilidades, facilitando el mantenimiento y permitiendo actualizaciones en una capa sin afectar las demás. Por ejemplo, cambiar la base de datos no impacta la lógica de negocio.

Ejemplo de Implementación: Capa de presentación con React, capa de lógica de negocio con servicios en Java, y capa de datos con un ORM como Hibernate conectado a PostgreSQL.

Ejercicios

Sistema de Reservas de Vuelos

Solución Necesaria: Una plataforma escalable para reservas de vuelos que integre búsqueda, pago y notificaciones.

Descripción del Problema: La plataforma debe soportar picos de tráfico, integrarse con múltiples proveedores externos (aerolíneas, pasarelas de pago) y permitir despliegues independientes.

Por qué usar Microservicios: Este patrón permite dividir la funcionalidad en servicios independientes (búsqueda de vuelos, gestión de pagos, notificaciones), cada uno con su propia base de datos y tecnología, lo que facilita la escalabilidad horizontal y el despliegue continuo.

Ejemplo de Implementación: Servicios en contenedores Docker (uno para búsqueda en Node.js, otro para pagos en Python), orquestados con Kubernetes.

Ejercicios

Sistema de Monitoreo de automatizadores de una fábrica

Solución Necesaria: Un sistema para procesar datos en tiempo real de dispositivos IoT, como sensores en una fábrica.

Descripción del Problema: Los dispositivos generan eventos continuos (temperatura, presión) que deben procesarse rápidamente para alertar sobre anomalías.

Por qué usar Event-Driven: Este patrón permite manejar flujos asíncronos de eventos, desacoplando productores (sensores) de consumidores (procesadores de datos), lo que mejora la escalabilidad y la capacidad de respuesta en tiempo real.

Ejemplo de Implementación: Uso de Apache Kafka para la cola de eventos, con microservicios consumiendo eventos para análisis y alertas.

Ejercicios

Sistema Operativo Personalizable

Solución Necesaria: Un sistema operativo ligero para dispositivos embebidos, con soporte para módulos personalizados.

Descripción del Problema: El sistema debe ser altamente modular, permitiendo agregar o quitar funcionalidades (como soporte para nuevos periféricos) sin recompilar todo el sistema.

Por qué usar Microkernel: El patrón Microkernel separa el núcleo del sistema (servicios básicos) de los módulos opcionales, permitiendo extensiones dinámicas y mejor mantenimiento.

Ejemplo de Implementación: Un núcleo minimalista que maneja procesos y memoria, con módulos como controladores de dispositivos cargados dinámicamente.

Ejercicios

Plataforma de Comercio Electrónico Solución Necesaria: Una tienda en línea que soporte picos de tráfico durante eventos como Black Friday.

Descripción del Problema: La aplicación debe manejar miles de transacciones simultáneas sin degradación del rendimiento.

Por qué usar Space-Based: Este patrón utiliza almacenamiento en memoria distribuida (como cachés) y procesamiento paralelo para manejar grandes volúmenes de datos y tráfico, eliminando cuellos de botella en bases de datos tradicionales.

Ejemplo de Implementación: Uso de Apache Ignite para caché distribuido, con nodos de procesamiento que manejan sesiones de usuario y carritos de compra.

Ejercicios

Sistema Microcreditos

Solución Necesaria: Un sistema para procesar transacciones bancarias, con auditorías y reportes regulatorios.

Descripción del Problema: El sistema requiere alta seguridad, trazabilidad y facilidad para actualizar reglas de negocio sin afectar la interfaz o el almacenamiento.

Por qué usar Capas: La separación en capas (presentación, lógica de negocio, datos) permite aislar la lógica de seguridad y reglas regulatorias, facilitando auditorías y actualizaciones.

Ejemplo de Implementación: Capa de presentación con Angular, lógica de negocio en Spring Boot, y almacenamiento en Oracle con auditoría en una capa separada.

Ejercicios

Plataforma de Streaming de Video

Solución Necesaria: Un sistema para streaming de video con recomendaciones personalizadas y facturación.

Descripción del Problema: La plataforma debe escalar servicios independientes (streaming, recomendaciones, pagos) y soportar múltiples regiones.

Por qué usar Microservicios: Permite que cada servicio (como el motor de recomendaciones) escale y se desarrolle de forma independiente, optimizando recursos y facilitando integraciones con terceros.

Ejemplo de Implementación: Servicio de streaming en Go, recomendaciones con TensorFlow en Python, y pagos en Java, coordinados con una API Gateway.

Ejercicios

Sistema de Procesamiento de Pagos

Solución Necesaria: Un sistema para procesar pagos en tiempo real, con detección de fraudes y notificaciones.

Descripción del Problema: Los pagos deben procesarse rápidamente, y las alertas de fraude deben generarse sin retrasar las transacciones.

Por qué usar Event-Driven: Este patrón permite que los eventos de pago desencadenen procesos paralelos (validación, detección de fraude, notificaciones), asegurando baja latencia y desacoplamiento.

Ejemplo de Implementación: Cola de eventos en RabbitMQ, con microservicios para validación de pagos y análisis de fraude en tiempo real.

Ejercicios

Sistema de Automatización Industrial

Solución Necesaria: Un software para controlar maquinaria industrial con soporte para nuevos dispositivos.

Descripción del Problema: El sistema debe ser extensible para integrar nuevos protocolos de comunicación y tipos de máquinas sin modificar el núcleo.

Por qué usar Microkernel: El núcleo proporciona servicios básicos (como comunicación y control), mientras que los módulos plug-in manejan dispositivos específicos, facilitando la extensibilidad.

Ejemplo de Implementación: Núcleo en C++ para control básico, con plug-ins para protocolos como Modbus o OPC UA.

Ejercicios

Plataforma de Análisis de Big Data

Solución Necesaria: Un sistema para analizar grandes volúmenes de datos en tiempo real, como logs de servidores.

Descripción del Problema: El sistema debe procesar terabytes de datos con baja latencia y alta disponibilidad.

Por qué usar Space-Based: Este patrón utiliza almacenamiento en memoria distribuida y particionamiento para procesar datos masivos en paralelo, garantizando escalabilidad y rendimiento.

Ejemplo de Implementación: Uso de Hazelcast para almacenamiento en memoria y Apache Spark para procesamiento distribuido.

Ejercicios

Plataforma de Mensajería Instantánea

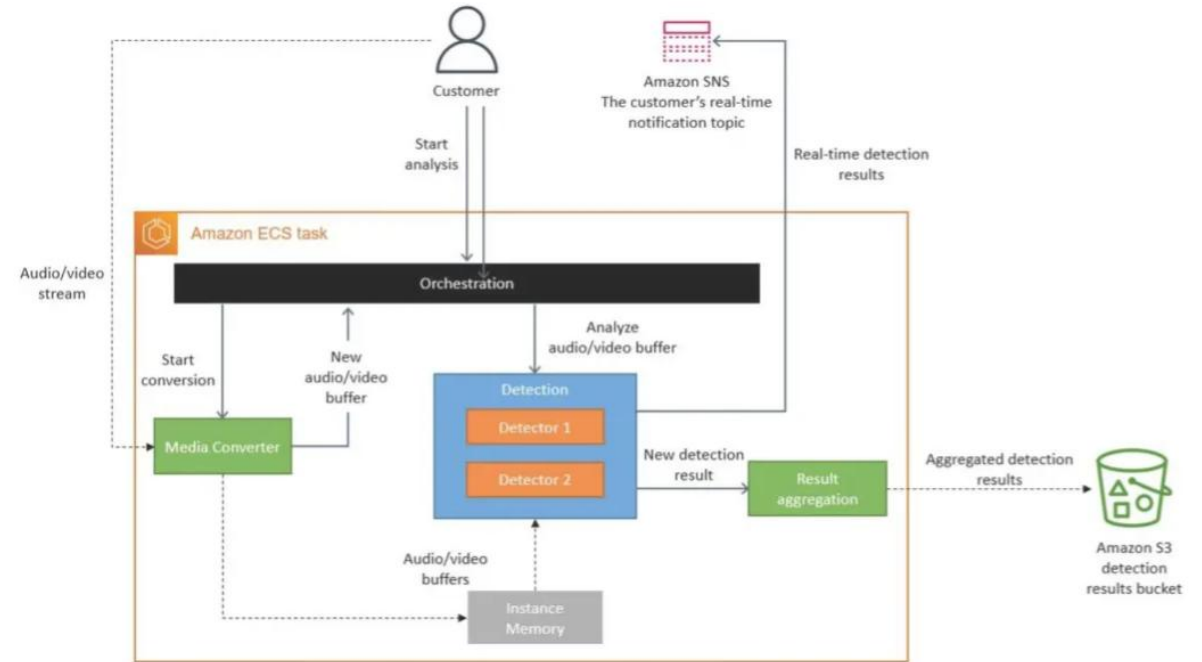
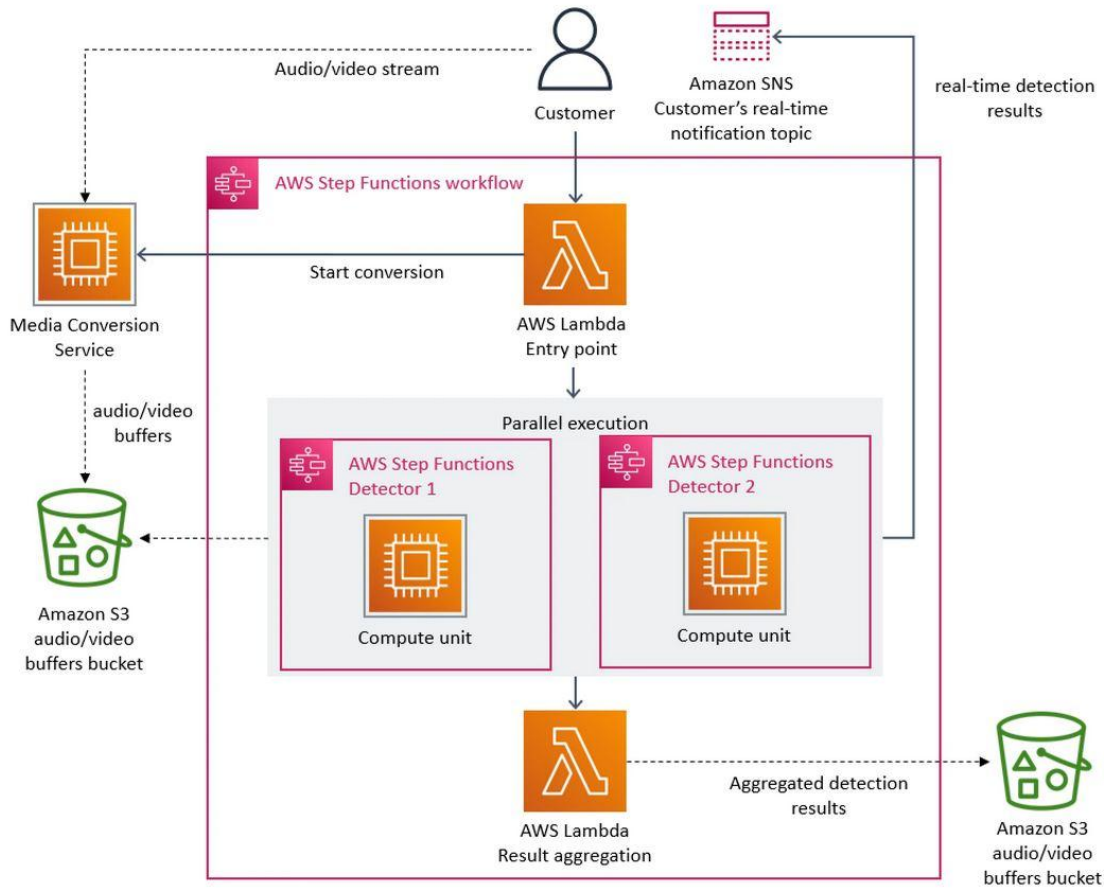
Solución Necesaria: Una aplicación de mensajería con soporte para chats, grupos y multimedia.

Descripción del Problema: La aplicación debe escalar para millones de usuarios y permitir actualizaciones frecuentes en funciones específicas.

Por qué usar Microservicios: Cada funcionalidad (chat, almacenamiento de multimedia, notificaciones) puede desarrollarse y escalarse de forma independiente, mejorando la agilidad y el rendimiento.

Ejemplo de Implementación: Servicios en Node.js para chats, almacenamiento en S3, y notificaciones con Firebase.

Caso Prime Video



Nueva arquitectura monolito en que todos los componentes se trasladan a un solo proceso.

ARQUITECTURAS COMPUESTAS

Digital businesses require **composable application architectures** that enable **agility, flexibility, integration and innovation**.

A mesh app and service architecture with an API-centric approach enables application architects and software engineers to meet those needs.



KEY FINDINGS

1



A **composable application architecture** is never finished. As business requirements and technologies evolve, so will the architecture.

2



A **mesh app and service architecture (MASA)** applies proven architecture patterns and intuitive user interface design to support an agile, composable architecture comprising multiexperience apps, mediated APIs and multigrained services.

3



Agile application architecture demands a distributed, decoupled approach that enables independent development, testing and deployment of each component. This approach allows developers to use the optimal technology for each component.



RECOMMENDATIONS

1



Use **MASA** to create an agile architecture in new applications. MASA enables high degrees of scalability, flexibility, and composability of applications and custom integrations.

2



Use **MASA** as the optimal approach to modernize legacy applications. The architecture allows you to update, replace and modernize components independent of each other at a rate your organization can tolerate.

3



Implement MASA in steps, incrementally building components and adapting functionality in each iteration. This allows you to evolve the architecture, hone your skills, and apply best practices as additional requirements for apps, APIs and services become clear.

4



Use a **mediated API approach** to abstract back-end service dependencies from multiexperience apps. Decompose applications into services as necessary to gain further agility.

5



Implement a versioning strategy for both components and interfaces that isolates the life cycle for each architectural component. This will minimize coupling and provide the flexibility you need to adapt and innovate.



EL VALOR DE UNA ARQUITECTURA
COMPUESTA (MASA)



Mayor agilidad
y velocidad



Flexibilidad para
adaptarse al cambio



Mejor integración
e innovación



Resiliencia y
confiabilidad



Equipos autónomos y
más productivos

COMPOSABLE ARCHITECTURE

MASA

Microservices + API + SaaS + AI

MASA es una arquitectura de aplicación composable, ágil y distribuida que permite a las organizaciones **crear, adaptar** y **modernizar** aplicaciones rápidamente.



¿QUÉ SIGNIFICA COMPOSABLE?



Composable Apps

Aplicaciones compuestas por módulos de negocio reutilizables que se pueden combinar, reemplazar o escalar de forma independiente.



APIs & Services

Servicios expuestos y consumidos a través de APIs que facilitan la integración, la orquestación y la reutilización.



Fit-for-purpose multiexperience apps

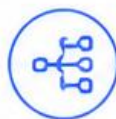
Experiencias diseñadas para canales y usuarios específicos, optimizadas para cada contexto de uso.

Outer APIs - Mediated APIs - Inner APIs



Outer APIs

Exponen capacidades a socios, clientes o ecosistemas.



Mediated APIs

Orquestan, combinan y adaptan servicios para necesidades específicas.



Inner APIs

Exponen capacidades internas de forma segura dentro de la organización.

Multigrained services

Servicios con diferentes niveles de granularidad (finos, medianos y gruesos) para maximizar la reutilización y el valor de negocio.



Servicios finos



Servicios medianos



Servicios gruesos



Empieza pequeño, evoluciona con el tiempo

Para implementar exitosamente MASA, comienza pequeño y evoluciona la arquitectura con el tiempo mientras desarrollas las habilidades necesarias, estableces coordinación y adaptas tu infraestructura.



¿Qué es MASA?

MASA es una arquitectura de aplicación composable, ágil y distribuida que aplica un enfoque modular para crear, adaptar y modernizar aplicaciones rápidamente.



La pregunta clave

“¿Cómo implemento una arquitectura de aplicación que proporcione la agilidad óptima para innovar y adaptarse al cambio?”



BENEFICIOS DE UNA ARQUITECTURA COMPOSABLE (MASA)



Mayor agilidad
e innovación



Escalabilidad
y flexibilidad



Menor costo
de cambio



Resiliencia
y disponibilidad



Equipos autónomos
y empoderados



Mejor experiencia
de cliente

COMPOSABLE ARCHITECTURE

20 HIDDEN BENEFITS

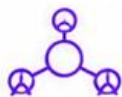
Composable Architecture no solo transforma la forma de construir software, sino también los **resultados del negocio**.

Estos beneficios suelen ser invisibles, pero su **impacto** es enorme.



20 BENEFICIOS OCULTOS DE LA COMPOSABLE ARCHITECTURE

1



Decentralized
Decision-Making

2



Easy
Maintainability

3



Enhanced Developer
Experience

4



Quick Decommissioning
Of Underperforming
Modules

5



Faster
Experimentation

6



Empowerment Of
Nontechnical Teams

7



Selective
Innovation

8



Rapid AI
Integration

9



Boosted Business
Flexibility

10



Readiness For Rapid
Strategic Pivots

11



More Independent
Teams

12



'Scope Creep'
Becoming 'Scope Leap'

13



The Ability To
Isolate Production Issues

14



Fast Innovation With
Minimal Disruption

15



Faster Developer
Onboarding

16



Avoidance Of
Vendor Lock-In

17



Freedom To
'Fail Cheaply'

18



Shorter
Time To Value

19



Rapid Disaster
Recovery

20



Reduced
'Cost Of Change'



BENEFICIOS QUE NO SIEMPRE SE VEN,
PERO QUE DEFINEN TU VENTAJA COMPETITIVA.



Más agilidad
para innovar



Mayor resiliencia
operacional



Decisiones más
rápidas



Equipos más
empoderados

Entregable 2

Modelamiento

Modelar la arquitectura de referencia y la arquitectura de implementación de la solución propuesta en el punto 1.

De la visión a un diseño concreto



Entregable 1
Caso de negocio
y requerimientos



Entregable 2
Modelamiento
(Arquitecturas)



Entregable 3
Validación y
plan de adopción



Una arquitectura bien modelada es la base para **soluciones sostenibles, escalables y alineadas con el negocio.**



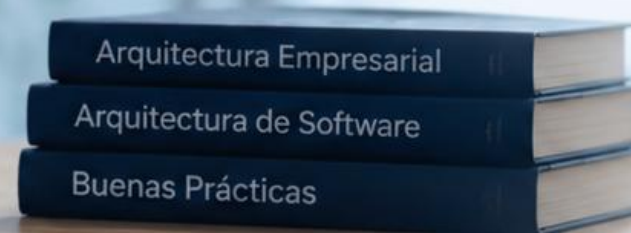
¿Qué **patrones arquitectónicos** utilizarás?



¿Cómo se relacionan los **componentes**?



¿Cómo soporta esta arquitectura los **requerimientos del negocio**?

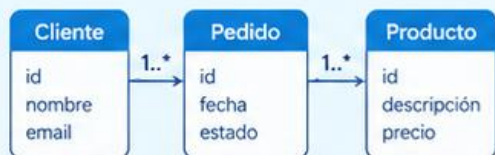


Proyecto Integrador – Entregables

Cada entregable aporta al desarrollo del proyecto y tiene un porcentaje de participación en la calificación total.

1 Modelo de dominio (completo) 5%

Diagrama que muestra las entidades del dominio, sus atributos, relaciones y reglas de negocio.



2 ADR (Architectural Decision Record) 25%

Documento que registra las decisiones clave de arquitectura, con su contexto, alternativas, motivación y consecuencias.



3 Arquitectura de referencia 20%

Vista de alto nivel de la solución, con sus capas, componentes, tecnologías y patrones.



4 Arquitectura de implementación 20%

Detalle técnico de cómo se despliegan los componentes, integraciones, infraestructura y configuraciones.



5 Diagrama de clases (de un caso) 5%

Representa las clases, atributos, métodos y relaciones para un caso específico del sistema.



6 Diagrama de secuencia (de un caso) 5%

Muestra la interacción entre objetos y el flujo de mensajes en un escenario específico.



7 Prototipo de interfaz de usuario 10%

Diseño funcional de las pantallas principales que muestran la experiencia del usuario (UI/UX).



8 Plataforma de observabilidad 5%

Monitoreo, métricas, trazas y logs para garantizar la operación, rendimiento y resiliencia del sistema.



9 Plan de pruebas unitarias sobre los casos de uso definidos. 5%

Estrategia, casos y resultados de pruebas unitarias que validan el correcto funcionamiento del código.



10 Definición de volumetría para pruebas 0%

Estimación de volúmenes de datos, usuarios y transacciones para escenarios de prueba.



11 Módulo de inyección de fallos 0%

Mecanismo para simular fallos y validar la resiliencia del sistema.



Un entregable, un objetivo:

Construir una solución de software sólida, escalable y alineada con las mejores prácticas de arquitectura y desarrollo.



Arquitectura
+ Calidad



Trabajo en equipo
+ Colaboración



Innovación
+ Mejores prácticas